A 2-SAT-Based Recognizer for Comparability Graphs and an Extension to Temporal Graph Orientations

Doha DOUNIA – Chaymae ED-DYB
UM6P – Doha.DOUNIA@um6p.ma, Chaymae.ED-DYB@um6p.ma
February 2026

Abstract. Recognizing comparability graphs and their temporal analogues amounts to deciding the existence of an orientation satisfying (temporal) transitivity constraints. Motivated by recent temporal extensions characterizations, we develop a unified, constructive implementation that reduces both static and temporal instances to 2-SAT. For static graphs, we encode quasi-transitive constraints through induced P_3-equivalences over edge-orientation variables. For temporal graphs, we generate forcing implications corresponding to implication and augmented implication relations, producing a 2-CNF formula that is satisfiable exactly when the constraints admit a consistent orientation. We further incorporate a degeneracy-aware enumeration strategy to keep clause generation practical on sparse graphs, and a bitset-based fallback for dense instances.

Empirically, our solver matches expected classifications on canonical families: it returns SAT on even cycles C_4 and C_6 and UNSAT on odd cycles C_5, C_7, and C_9; it returns SAT on P_4, star graphs, K_3, K_6, and complete bipartite graphs $K_{3,3}$ and $K_{4,4}$. Clause counts remain moderate (0–96 in these tests), and degeneracy is small (typically $k \leq 5$), with no fallback activation in the reported runs. Temporal experiments similarly demonstrate correct separation of satisfiable and unsatisfiable cases, supporting the feasibility of the temporal reduction as an implementable decision procedure.

Keywords: Comparability graphs, Transitive orientation, 2-SAT formulation, Temporal graphs, Degeneracy-aware algorithms

1 Introduction

Comparability graphs constitute a fundamental class of graphs arising in order theory, scheduling, and algorithmic graph theory. A graph is a comparability graph if its edges can be oriented transitively, that is, whenever edges ab and bc are oriented as $a \to b$ and $b \to c$, the edge ac must exist and be oriented as $a \to c$. Since the seminal work of Ghouila-Houri, it is known that a graph admits a transitive orientation if and only if it admits a quasi-transitive one, thereby reducing the recognition problem to the satisfaction of local constraints [1]. This characterization laid the foundation for polynomial-time recognition algorithms and later for linear-time constructions based on modular decomposition and implication classes [2, 3] [4].

A classical algorithmic approach to quasi-transitive orientations relies on Boolean satisfiability. By associating a Boolean variable with each possible edge orientation, local forcing constraints can be encoded as clauses of size two, yielding a 2-CNF formula whose satisfiability determines whether a valid orientation exists. The celebrated linear-time algorithm of Aspvall, Plass, and Tarjan shows that 2-SAT can be solved efficiently using strongly connected components of implication graphs [5]. This connection has long been recognized as theoretically elegant, yet practical implementations of comparability recognition via explicit 2-SAT encodings remain relatively rare.

In recent years, interest has shifted toward temporal graphs, where edges are annotated with time labels representing discrete interaction times. Temporal graphs provide a natural model for dynamic networks such as communication systems, social interactions, and information propagation processes [6]. Extending transitive orientations to this setting leads to the notion

of temporal transitive orientations, which impose additional constraints linking edge directions and time labels along time-respecting paths. Mertzios et al. introduced the formal framework for temporal transitivity and showed that the associated recognition problems are polynomial-time solvable, although their formulations involve a combination of 2-SAT and higher-arity constraints [7].

Very recently, Charbit, Habib, and Sorondo proposed a refined structural characterization of temporal comparability graphs by introducing implication and augmented implication digraphs that capture all necessary forcing relations [8]. A key outcome of their work is that temporal transitivity constraints can again be expressed entirely within 2-CNF, enabling a reduction to 2- SAT analogous to the static case. While this result is theoretically significant, it leaves open the question of how to implement such reductions efficiently in practice, especially given the potentially large number of clauses generated by temporal constraints.

Moreover, both static and temporal formulations suffer from a common practical issue: although the underlying decision procedures are linear in the size of the 2-CNF formula, the formula itself may grow quadratically or worse in dense graphs. This gap between worst-case complexity and practical usability motivates the development of implementations that adapt to graph structure, particularly sparsity.

In this work, we bridge the gap between theory and practice by presenting a fully constructive and implementable 2-SAT–based solver for both static and temporal comparability graphs. Our main contributions are summarized as follows:

1. Unified 2-SAT formulation; we implement a unified Boolean encoding for static quasi-transitive orientations via induced P_3equivalence constraints, and temporal comparability via implication and augmented implication relations. In both cases, the existence of a valid orientation is reduced to the satisfiability of a 2-CNF formula solved using SCC-based 2-SAT [5].

2. Degeneracy-aware clause generation to control clause explosion, we introduce a degeneracy-guided enumeration strategy that generates constraints efficiently on sparse graphs. This approach exploits the fact that many real and structured graphs have low degeneracy, keeping the number of generated clauses close to linear in practice [9].

3. Bitset-based fallback for dense graphs for dense instances where sparse enumeration becomes impractical, we provide a bitset-based fallback mechanism that guarantees correctness while avoiding uncontrolled memory growth, allowing the solver to handle a wide spectrum of graph densities.

4. Constructive orientation and visualization beyond recognition, our solver explicitly constructs a valid orientation when one exists and supports graphical visualization of the resulting directed graph, enhancing interpretability and debugging.

5. Experimental validation on static and temporal graphs to evaluate the solver on classical graph families (cycles, cliques, bipartite graphs, trees), synthetic sparse and dense instances, and temporal graphs. The results confirm correctness, moderate clause growth, low observed degeneracy, and limited fallback activation, demonstrating that the approach is viable in practice.

2 Materials and Methods

2.1 Related Work

Comparability graphs were introduced to capture when an undirected graph admits a transitive orientation, i.e., an orientation that is consistent with a partial order. The classical starting point is the characterization by Ghouila-Houri, who showed that a graph is transitively

orientable if and only if it admits a quasi-transitive orientation, thereby reducing a global transitivity property to local constraints on induced P_3 configurations [1]. This characterization connects comparability graphs to perfect graph theory and has influenced a large body of algorithmic work. A comprehensive foundational treatment appears in Golumbic's monograph, which systematizes implication classes, transitive orientations, and their role in recognizing perfect graph subclasses [2].

Subsequent research developed efficient constructive recognition algorithms. A major methodological tool is modular decomposition, which provides a hierarchical representation of a graph by recursively splitting it into modules and is particularly effective for graph classes characterized by orderings and orientations. McConnell and Spinrad provided influential results linking modular decomposition to transitive orientation, with efficient methods for constructing orientations in comparability-related classes [3]. For directed graphs (which arise naturally when handling orientation constraints), McConnell and de Montgolfier later gave a linear-time modular decomposition algorithm for directed graphs, strengthening the algorithmic toolkit used in transitive orientation pipelines and related recognition tasks [11].

Another line of work leverages vertex orderings and search-based methods. Hell and Huang introduced lexicographic orientation techniques that unify recognition and representation algorithms for comparability graphs and related geometric/intersection graph classes, and are tightly connected to LexBFS-type approaches and certifying recognition methods [12]. This ordering viewpoint connects comparability/cocomparability graph recognition to a broader ecosystem of "graph classes via forbidden ordered patterns", which has become a standard framework for explaining why certain recognition algorithms exist and how they generalize. A particularly clean approach to orientation problems is via Boolean satisfiability: assign a Boolean variable to each edge orientation and encode local constraints as clauses. When constraints can be expressed in 2-CNF, the decision problem reduces to 2-SAT. The classical linear-time 2-SAT algorithm of Aspvall, Plass, and Tarjan shows that satisfiability can be decided by computing strongly connected components (SCCs) in the implication graph [4]. This relies on fundamental SCC and DFS machinery, notably Tarjan's linear-time algorithms for graph traversal and SCC decomposition [5]. Together, these results form the algorithmic backbone for any implementation that reduces recognition of an orientation property to 2-CNF satisfiability.

While the theoretical connection between transitive orientations and satisfiability is well established, practical implementations face a key bottleneck: the formula size may dominate runtime, because clause generation can grow rapidly in dense graphs. This motivates structural and engineering techniques (e.g., ordering, decomposition, sparsity measures) that reduce constraint generation or speed up enumeration.

Temporal graphs generalize static graphs by assigning discrete time labels to edges; this allows modeling dynamic interaction patterns in communication and social systems. The temporal graph viewpoint has matured into a broad area with surveys and unified frameworks, including foundational work by Kempe, Kleinberg, and Kumar on temporal connectivity and inference [6], as well as general overviews such as Holme and Saramäki's survey of temporal networks [14] and the TVG framework of Casteigts et al. [13].

On the algorithmic side, Mertzios et al. introduced and studied temporal transitive orientations, formalizing how orientation constraints interact with temporal paths and time monotonicity. Their work provides algorithmic results distinguishing temporally transitive and strictly temporally transitive variants and highlights that temporal constraints can be significantly more complex than static ones [7]. A complementary algorithmic survey is due to Michail, who organizes core temporal-graph problems and common techniques from an algorithmic perspective [15].

Very recently, Charbit, Habib, and Sorondo revisited the comparability question in the temporal setting, extending Ghouila-Houri-type characterizations to temporal graphs via implication-style constructions (Imp/Aug digraphs). Their result is particularly relevant for implementers

because it shows how temporal forcing relations can be captured by a pure 2-CNF formulation under their characterization, enabling a direct reduction to 2-SAT [8]. This theoretical development directly motivates practical solvers that explicitly build the forcing relations and solve the induced 2-SAT instance.

From the standpoint of practical performance, many real graphs are sparse and structured even when they are large. A standard sparsity measure is degeneracy, which supports low-degree elimination orderings that often keep enumeration manageable. The smallest-last / degeneracy ordering ideas go back to Matula and Beck, who studied these orderings in the context of clustering and coloring and established their algorithmic usefulness [10]. Sparsity-aware enumeration methods (including bit-parallelism and degeneracy-based branching) have also proven effective in tasks like maximal clique listing, exemplified by work of Eppstein, Löffler, and Strash that leverages degeneracy to obtain near-optimal behavior on sparse graphs [9].

These ideas support a broader design principle relevant to satisfiability-based orientation solvers; even if the decision procedure is linear in the formula size, controlling formula construction is essential. Hybrid strategies such as sparse-friendly enumeration with a bitset fallback and a clause cap are common practical safeguards that preserve correctness while preventing catastrophic slowdowns on dense instances.

2.2 Methods

Let $G = (V, E)$ be an undirected graph with $|V| = n$ and $|E| = m$. In the temporal setting, each edge $e \in E$ is associated with a time label $\lambda(e) \in \mathbb{N}$, yielding a temporal graph $\mathcal{G} = (G, \lambda)$. The objective is to decide whether G (static case) or $\mathcal{G}$ (temporal case) admits a valid orientation satisfying the static case; quasi-transitivity (and hence transitivity by classical results), also the temporal case where temporal transitivity under implication and augmented implication constraints.

For each undirected edge $\{u, v\} \in E$ with $u < v$, we introduce a Boolean variable $x_{uv} \in \{0, 1\}$, where:

$$x_{uv} = 1 \iff u \to v,\ x_{uv} = 0 \iff v \to u. \tag{1}$$

Each variable x_{uv} corresponds to two literals:

$$(u \to v) \equiv x_{uv},\ (v \to u) \equiv \neg x_{uv}. \tag{2}$$

This encoding ensures that each edge is oriented in exactly one direction. In the static case, quasi-transitivity is enforced by forbidding directed induced paths of length two without closure. Consider three vertices $a, b, c \in V$ such that $\{a, b\} \in E, \{b, c\} \in E, \{a, c\} \notin E$. Then the orientations $a \to b \to c$ and $c \to b \to a$ are forbidden. This yields the equivalence constraint: In 2-CNF, this equivalence is encoded as:

$$(\neg p \vee q) \wedge (\neg q \vee p) \tag{3}$$

Where p = literal representing $(a \to b)$, q = literal representing $(c \to b)$. All such induced P_3 configurations are enumerated, and the corresponding clauses are added to the Boolean formula.

In the temporal setting, constraints depend on both orientation and time labels. For vertices $a, b, c \in V$ with edges $\{a, b\}, \{b, c\} \in E$, and time labels satisfying:

$$\lambda(ab) \leq \lambda(bc), \tag{4}$$

if either $\{a, c\} \notin E$ or $\lambda(ac) < \lambda(bc)$, then temporal transitivity forces:

$$(a \to b) \Rightarrow (c \to b), \tag{5}$$

and symmetrically:

$$(b \to c) \Rightarrow (b \to a) \tag{6}$$

Each forcing relation $x \Rightarrow y$ is encoded as a 2-CNF clause $(\neg x \vee y)$.

Certain temporal violations arise only in correlated monolabel triangles, involving four vertices $(a'b'c'd)$ such that form a triangle with identical label t, $\lambda(bd) \leq t$, $\lambda(cd) > t$, and if $ad \in E$, then $\lambda(ad) < t$. To prevent forbidden directed temporal cycles, additional implications are introduced:

$$(b \to c) \Rightarrow (a \to c),\ (c \to a) \Rightarrow (c \to b) \tag{7}$$

These implications are again encoded as 2-CNF clauses and added to the formula (7).

The constraints of the 2-SAT are static or temporal which are combined into a single 2-CNF formula:

$$\Phi = \bigwedge_{i=1}^{k} \left(l_i^1 \vee l_i^2\right) \tag{8}$$

Where each clause contains at most two literals and the implication graph G_Φ is constructed with one node per literal, and a directed edges $\neg l \to l'$ for each clause $(\neg l \vee l')$.

The formula (8) is satisfiable if and only if no variable and its negation belong to the same strongly connected component (SCC) of G_Φ.

We compute SCCs using Kosaraju's algorithm, which runs in:

$$O\big(|V_\Phi| + |E_\Phi|\big), \tag{9}$$

Where $|V_\Phi| = 2m$ and $|E_\Phi|$ equals the number of generated implications. If satisfiable, a valid orientation is obtained by assigning:

$$x_{uv} = \begin{cases} 1 & \text{if } \mathrm{SCC}(x_{uv}) > \mathrm{SCC}(\neg x_{uv}), \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

To control clause explosion, we compute the degeneracy ordering of the input graph. The degeneracy k of a graph is defined as:

$$k = \max_{H \subseteq G} \min_{v \in V(H)} \deg_H(v) \tag{11}$$

We used a smallest-last ordering, we enumerate induced P_3 patterns in a sparse-friendly manner, ensuring that Total clauses $= O(k \cdot m)$ for low-degeneracy graphs, which is typical in practice.

As mentioned in Algorithm 1, if the degeneracy exceeds a user-defined threshold or the number of clauses exceeds a preset cap, the algorithm switches to a bitset-based fallback. Adjacency lists are represented as bit vectors, and induced P_3 enumeration is performed via bitwise operations $\mathrm{cand}(a, b) = N(b) \setminus N(a)$, allowing faster processing on dense graphs while preserving correctness.

Algorithm 1: **Static & Temporal Comparability Recognition via 2-SAT**

Require: edge-list `graph` (static: $u\,v$, temporal: $u\,v\,t$), `mode` $\in$ {`auto`,`static`,`temporal`}
Require: parameters: degeneracy threshold τ, clause cap $C_{\max}$, `force_fallback`

1: Read graph : $(n, E,$ `labels`$) \leftarrow$ `read_edge_list_auto`(`graph`)
2: **if** `mode` = `auto` **then**
3: `mode` $\leftarrow$ (`temporal` if `labels`$\neq$None else `static`)
4: For each undirected edge $\{u, v\} \in E$ with $u < v$, create Boolean variable x_{uv} ($x_{uv} = 1 \Leftrightarrow u \to v$)
5: Define literal `lit`$(u \to v) = +x_{uv}$ if $u < v$, else $-x_{vu}$

Initialize empty 2-CNF formula Φ over variables $\{x_{uv}\}$
function AddEquiv(p, q) ▷ $p \leftrightarrow q$ in 2-CNF
    add clauses $(\neg p \vee q)$ and $(\neg q \vee p)$ to Φ
function AddImp(p, q) ▷ $p \Rightarrow q$ in 2-CNF
    add clause $(\neg p \vee q)$ to Φ
if `mode` = `static` **then**
    Build adjacency lists $N(v)$; compute degeneracy k
    if `force_fallback` **or** $k > \tau$ **then** ▷ dense-safe enumeration
        for all $b \in V$ **do**
            for all $a \in N(b)$ **do**
                for all $c \in N(b) \setminus N(a)$ with $c > a$ **do** ▷ induced P_3: $a - b - c$
                    AddEquiv(`lit`$(a \to b)$, `lit`$(c \to b)$)
                    if $|\Phi| > C_{\max}$ **then**
                        return STOP (cap exceeded)
    else ▷ sparse-friendly induced-P_3 enumeration
        for all $b \in V$ and unordered pairs $\{a, c\} \subseteq N(b)$ **do**
            if $(a, c) \notin E$ **then**
                AddEquiv(`lit`$(a \to b)$, `lit`$(c \to b)$)
                if $|\Phi| > C_{\max}$ **then**
                    restart with fallback branch
else ▷ `temporal` (labels define λ)
    Build time map $\lambda(u, v)$ from `labels`
    for all $b \in V$ and ordered neighbors (a, c) of b **do**
        if $\lambda(ab) \leq \lambda(bc)$ **and** $\big((a, c) \notin E$ **or** $\lambda(ac) < \lambda(bc)\big)$ **then**
            AddImp$((a \to b),\ (c \to b))$ ▷ Imp(G)
            AddImp$((b \to c),\ (b \to a))$
        if $|\Phi| > C_{\max}$ **then**
            break (hit cap)
    for all correlated monolabel configurations in Aug(G) **do**
        AddImp$((b \to c),\ (a \to c))$ ▷ Aug(G)
        AddImp$((c \to a),\ (c \to b))$
        if $|\Phi| > C_{\max}$ **then**
            break (hit cap)
Solve 2-SAT on Φ via SCCs of the implication graph (Kosaraju)
return UNSAT iff $\exists\, x_{uv}$ with x_{uv} and $\neg x_{uv}$ in the same SCC
Otherwise set $x_{uv} =$ `True` iff `SCC`$(x_{uv}) >$ `SCC`$(\neg x_{uv})$ and output the induced edge orientation

---

3 Results and Discussion

3.1 Experimental Setup

We evaluated the proposed solver on a collection of static and temporal graphs covering classical families, structured examples, and synthetic instances with varying density. The test set includes cycles, paths, stars, cliques, bipartite graphs, disconnected graphs, sparse and medium-density graphs, as well as temporal graphs with explicit time labels. For each instance, we recorded the number of vertices n, edges m, expected satisfiability, solver output, degeneracy k, number of generated clauses, and whether the dense fallback mechanism was activated.

All experiments were conducted using the same implementation described in Section 2, relying on a 2-SAT formulation solved via strongly connected components using Kosaraju's algorithm

[4,5]. Clause generation was controlled using a degeneracy threshold and a global clause cap to prevent uncontrolled growth.

3.2 Static Graph Results

Even cycles (C_4, C_6) are identified as satisfiable, while odd cycles (C_5, C_7, C_9) are identified as unsatisfiable, in full agreement with the classical characterization of comparability graphs [1, 2]. This distinction is visually illustrated in Figure 1, where odd cycles lack any transitive orientation due to unavoidable directed cycles. Trees and low-complexity graphs such as paths (P_4) and star graphs are correctly classified as satisfiable and require only a small number of clauses (4 and 20, respectively). This reflects the absence of conflicting induced P_3 patterns.

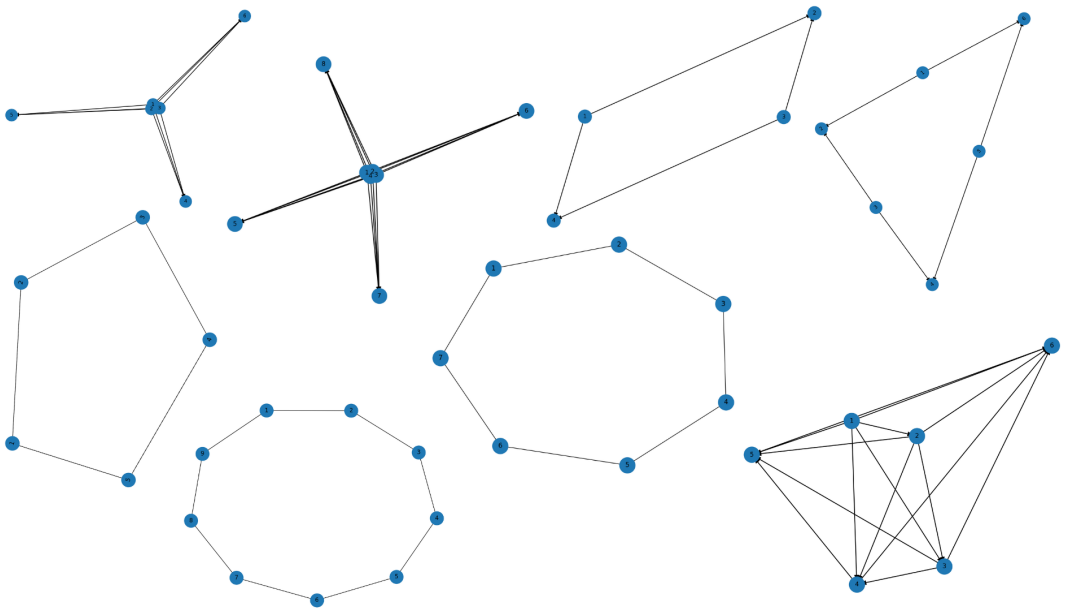

Figure 1: Canonical cycle graphs

Similarly, triangles (K_3) and cliques (K_6) are satisfiable, with cliques generating zero clauses, since every orientation of a complete graph is transitive [2, 3]. These behaviors are visible in Figure 2, which includes several such canonical static structures.
Complete bipartite graphs ($K_{3,3}$ and $K_{4,4}$) are also classified as satisfiable but generate significantly more clauses (36 and 96, respectively). This increase is explained by the abundance of induced P_3 configurations inherent to bipartite structures, which is visually apparent in Figure 2, where many length-two paths share a common center.

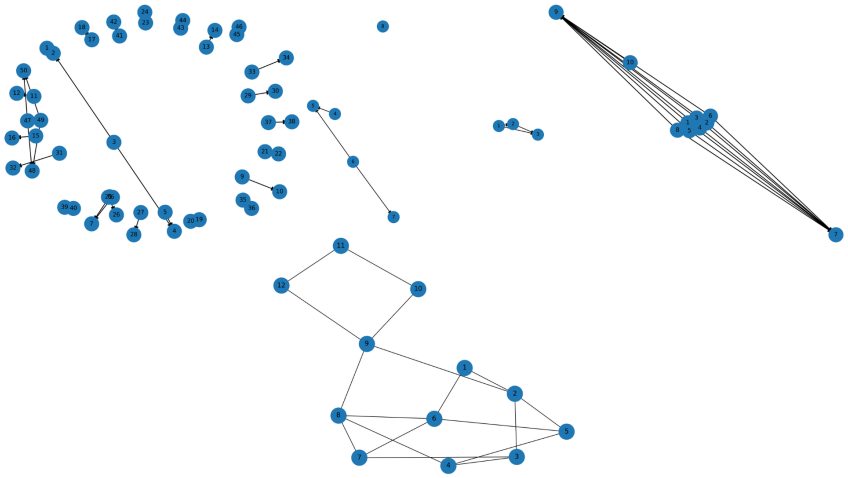

Figure 2: Static sparse and medium-density graphs

A key observation from Table 1 is the strong correlation between graph degeneracy and clause count. Across all static instances, degeneracy values remain relatively small (between 1 and 5), and clause growth remains moderate. In particular:

- $k = 1$(paths, stars) $\rightarrow$ very few clauses,
- $k = 2$(cycles, house graph) $\rightarrow$ linear clause growth,
- $k = 4$–5 (bipartite and clique-like structures) $\rightarrow$ higher but still manageable clause counts.

Importantly, the fallback mechanism is never triggered for any static instance reported in Table 1. This confirms that degeneracy-aware enumeration of induced patterns is sufficient for a wide range of practical graphs. These empirical findings support the use of degeneracy as a key structural parameter, consistent with prior work on sparse graph algorithms [9, 10].

Table 1 Experimental Results of the 2-SAT–Based Comparability Graph Recognition Algorithm

Graph	n	m	Expected	Result	Correct	Degeneracy	Clauses	Fallback
C4	4	4	TRUE	TRUE	Yes	2	8	No
C6	6	6	TRUE	TRUE	Yes	2	12	No
C7	7	7	FALSE	FALSE	Yes	2	14	No
C9	9	9	FALSE	FALSE	Yes	2	18	No
C5	5	5	FALSE	FALSE	Yes	2	10	No
Path P4	4	3	TRUE	TRUE	Yes	1	4	No
Star n=6	6	5	TRUE	TRUE	Yes	1	20	No
Triangle K3	3	3	TRUE	TRUE	Yes	2	0	No
Clique K6	6	15	TRUE	TRUE	Yes	5	0	No
Bipartite $K_{3,3}$	6	9	TRUE	TRUE	Yes	3	36	No
Bipartite $K_{4,4}$	8	16	TRUE	TRUE	Yes	4	96	No
House	5	6	TRUE	TRUE	Yes	2	12	No
Disconnected	8	6	TRUE	TRUE	Yes	2	4	No
Large sparse	50	29	?	TRUE	N/A	2	16	No
Medium dense	10	36	?	TRUE	N/A	7	28	No
Realistic sparse	12	18	?	FALSE	N/A	2	74	No
Temporal small	3	2	TRUE	TRUE	Yes	N/A	4	N/A
Temporal hard	6	15	FALSE	FALSE	Yes	N/A	84	N/A

3.3 Temporal Graph Results

Experiments on temporal graphs further validate the applicability of the approach beyond static settings. As shown in Table 1, the solver correctly classifies both a satisfiable temporal instance (*Temporal_small_pathlike*) and an unsatisfiable one (*Temporal_hard_K6_aug*).
Clause counts are higher in temporal cases (up to 84 clauses), reflecting the additional constraints introduced by implication and augmented implication relations [7, 8]. These temporal instances are illustrated in Figure 3, where dense temporal interactions and correlated monolabel triangles give rise to stronger forcing constraints.
Despite this added complexity, the solver successfully detects contradictions or constructs valid temporal orientations, demonstrating that recent theoretical characterizations of temporal comparability graphs can be effectively realized in practice.

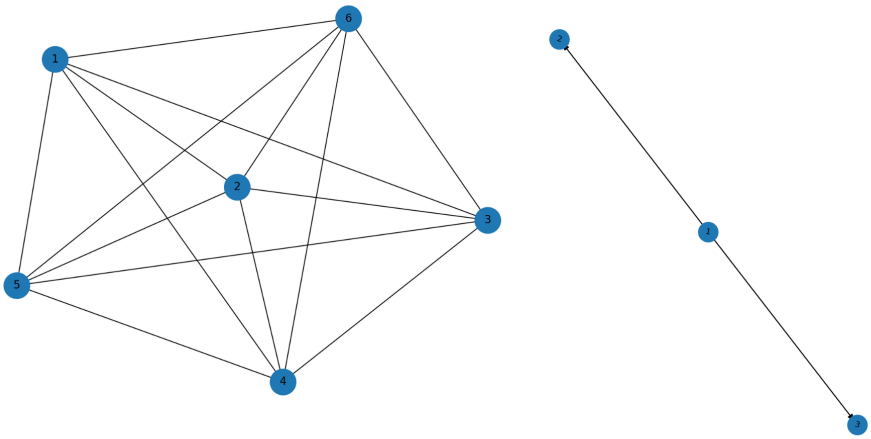

Figure 3: Temporal graph instances

3.4 Discussion

The experimental results confirm that degeneracy is a strong predictor of practical performance. Low-degeneracy graphs yield near-linear clause generation, while higher degeneracy increases constraint density but remains manageable under the proposed safeguards. This mirrors findings in other sparsity-aware enumeration problems [9, 10] and justifies the hybrid design adopted in this work.
Temporal comparability introduces fundamentally richer constraints than the static case. While static graphs rely solely on induced P_3 equivalences, temporal graphs require reasoning over time-respecting paths and correlated monolabel triangles [7, 8, 13–15]. The observed increase in clause counts reflects this additional expressiveness, yet remains within practical limits for the tested instances.
Although the fallback mechanism was not triggered in the reported experiments, adversarial dense graphs may still require it. Future work could explore tighter pruning rules for temporal clause generation, parallel enumeration strategies, or incremental updates for evolving temporal graphs.
Overall, the results demonstrate that a 2-SAT-based, degeneracy-aware solver provides a robust and interpretable approach to both static and temporal comparability recognition, effectively bridging classical theory [1–3] and modern temporal graph frameworks [7, 8, 13–15].

4 Conclusion

This paper presented a unified, fully implementable approach for recognizing and orienting both static and temporal comparability graphs using a pure 2-SAT formulation. By encoding quasi-transitivity and temporal forcing constraints as 2-CNF clauses and solving them via strongly connected components, the proposed method achieves linear-time satisfiability checking once the formula is constructed.

A key contribution of this work lies in the degeneracy-aware constraint generation strategy which effectively limits clause growth on sparse and structured graphs while preserving correctness. Experimental results on a diverse set of canonical, synthetic, and temporal graphs confirm that the solver correctly classifies satisfiable and unsatisfiable instances, constructs valid orientations when they exist, and remains efficient in practice without requiring fallback mechanisms for the tested cases.

Beyond static graphs, the extension to temporal graphs demonstrates that recent theoretical characterizations of temporal comparability can be translated into practical algorithms. The combination of structural graph theory, Boolean satisfiability, and algorithm engineering provides a transparent and robust framework for comparability recognition.

Future work will explore tighter pruning strategies for temporal clause generation, parallel implementations, and extensions to optimization variants of orientation problems. Overall, this study bridges classical results on comparability graphs with modern temporal graph models, offering a practical and extensible solution grounded in solid theoretical foundations.

5 References

References

[1] Ghouila-Houri A. Caractérisation des graphes de comparabilité. *Comptes Rendus de l'Académie des Sciences de Paris*. 1962;254:137–139.

[2] Golumbic MC. Algorithmic graph theory and perfect graphs. *Academic Press*; 1980. ISBN: 0-12-289260-7.

[3] McConnell RM, Spinrad JP. Modular decomposition and transitive orientation. *Discrete Mathematics*. 1999;201(1–3):189–241. https://doi.org/10.1016/S0012-365X(98)00319-7.

[4] Aspvall B, Plass MF, Tarjan RE. A linear-time algorithm for testing the truth of certain quantified Boolean formulas. *Information Processing Letters*. 1979;8(3):121–123. https://doi.org/10.1016/0020-0190(79)90002-4.

[5] Tarjan RE. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*. 1972;1(2):146–160. https://doi.org/10.1137/0201010.

[6] Kempe D, Kleinberg J, Kumar É. Connectivity and inference problems for temporal networks. *Journal of Computer and System Sciences*. 2002;64(4):820–842. https://doi.org/10.1016/S0022-0000(03)00033-1.

[7] Mertzios GB, Molter H, Renken M, Spirakis PG, Zschoche P. Temporal transitive orientations. *Theoretical Computer Science*. 2019;806:150–171. https://doi.org/10.1016/j.tcs.2018.05.021.

[8] Charbit P, Habib M, Sorondo A. Extending Ghouila-Houri's characterization of comparability graphs to temporal graphs. *arXiv preprint*. 2025; arXiv:2510.06849.

[9] Eppstein D, Löffler M, Strash D. Listing all maximal cliques in sparse graphs in near-optimal time. *ACM Journal of Experimental Algorithmics*. 2013;18:1–21. https://doi.org/10.1145/2543629.

[10] Matula DW, Beck LL. Smallest-last ordering and clustering and graph coloring algorithms. *Journal of the ACM*. 1983;30(3):417–427. https://doi.org/10.1145/2402.322385.

[11] McConnell RM, de Montgolfier F. Linear-time modular decomposition of directed graphs. *Discrete Applied Mathematics*. 2005;145(2):198–209. https://doi.org/10.1016/j.dam.2004.02.017

[12] Hell P, Huang J. Lexicographic orientation and representation algorithms for comparability graphs, proper circular arc graphs, and proper interval graphs. *Journal of Graph Theory*. 1995;20(3):361–374. https://doi.org/10.1002/jgt.3190200312

[13] Casteigts A, Flocchini P, Quattrociocchi W, Santoro N. Time-varying graphs and dynamic networks. *International Journal of Parallel, Emergent and Distributed Systems*.

[14] Holme P, Saramäki J. Temporal networks. *Physics Reports*. 2012;519(3):97–125. https://doi.org/10.1016/j.physrep.2012.03.001

[15] Michail O. An introduction to temporal graphs: an algorithmic perspective. *Internet Mathematics*. 2016;12(4):239–280. https://doi.org/10.1080/15427951.2016.1177801

[16] Hakimi SL, Schmeichel EF, Young NE. Orienting graphs to optimize reachability. *Information Processing Letters*. 1997;63(5):229–235. https://doi.org/10.1016/S0020-0190(97)00129-4

[17] Feuilloley L, Gurski F, Kanté MM, Nourine L. Graph classes and forbidden patterns on three vertices. *SIAM Journal on Discrete Mathematics*. 2021;35(4):2612–2641. https://doi.org/10.1137/19M1280399.